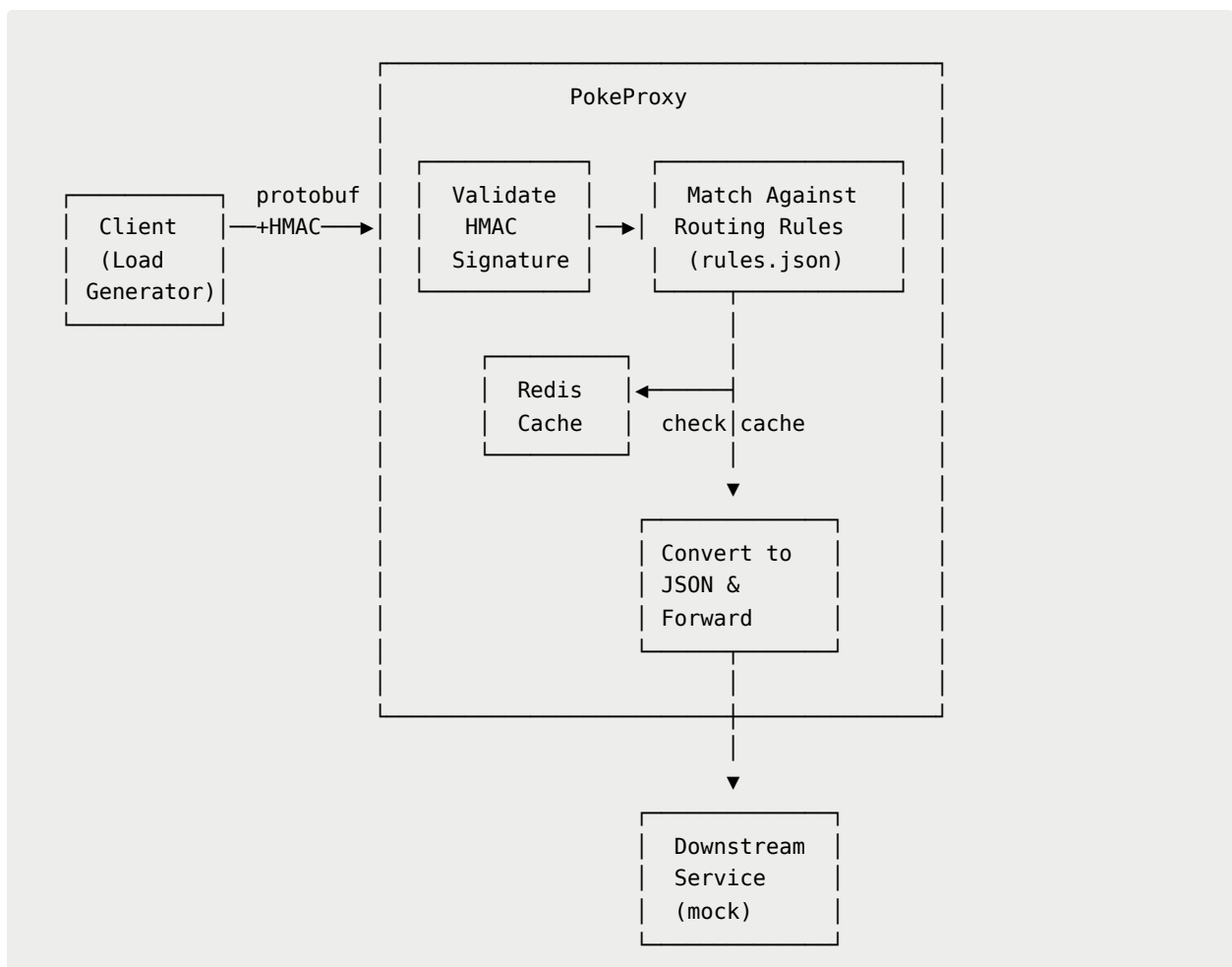


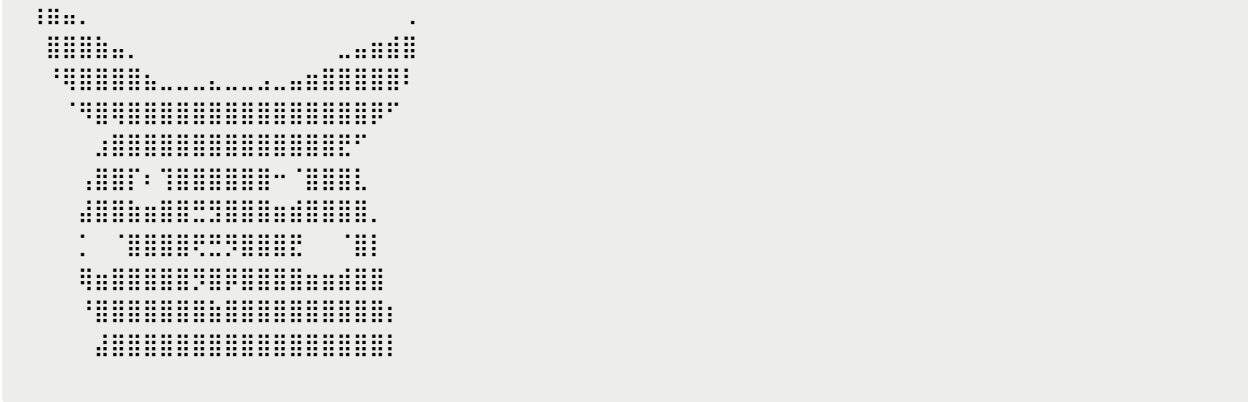


DevOps Home Assignment — PokeProxy

Background

PokeProxy is a reverse proxy service that receives Pokemon data streams as protobuf-encoded payloads, validates HMAC signatures, matches against configurable routing rules, and forwards matching Pokemon as JSON to downstream services. It includes a Redis caching layer to avoid re-processing previously seen payloads.





The service has been running in a development environment but has never gone through proper production engineering.

You are the first DevOps engineer joining the team. Your job is to take this service from "runs on a laptop" to "ships to production on every merge": fix what blocks reliable operation, containerize and deploy it, build the delivery pipeline, instrument it with monitoring, and automate everything you'd otherwise do twice.

Time Limit & Submission

You have the time limit specified in the email you received. Please submit your work as a **ZIP file** or a **GitHub repository link**.

What You're Given

- The `app/` directory contains the full application source code
- A mock downstream service is included for local testing
- A load generator script sends synthetic Pokemon traffic
- See `app/README.md` for details on how the application works

The Assignment

Part 1 — Code Review & Production Hardening

Review the application code from an operations perspective. You're looking for issues that would impact reliability, deployability, and operability in production.

- Identify and fix reliability issues in the code

- For each issue you find, write it up as a documented "issue" — describe the problem, its production impact, and your solution. Don't just fix it in the code; we want to see how you communicate findings to a team
- Beyond fixing bugs, **make the code deployable and operable**: configuration and secrets hygiene, structured logging, graceful shutdown, useful error messages, and any observability or operability improvements you think the service needs. The code is yours to change freely

Part 2 — Infrastructure & Deployment

Deploy the service on a local Kubernetes cluster.

- Containerize the application (write a Dockerfile)
- Create Kubernetes manifests (or a Helm chart / Kustomize overlay) to deploy:
 - The PokeProxy application
 - The mock downstream service
 - A Redis instance for the caching layer
- The service should be accessible and healthy within the cluster
- Apply production best practices: resource limits, health probes, sensible configuration, secrets handled properly

Part 3 — CI/CD & GitOps

Time-to-production is our favorite metric. Build the delivery pipeline that takes a code change from commit to running in the cluster.

- Create a CI pipeline (we use GitHub Actions, but use what you prefer) that lints/tests the code and builds the container image
- Define the CD side: how a new image version reaches the cluster. A GitOps approach (Argo CD-style — desired state in git, cluster converges to it) is encouraged, but a well-reasoned scripted deploy is also fine
- Add a **post-deploy verification step**: an automated end-to-end check that proves the deployed service actually works (sends real traffic through the proxy and validates the result) and gates the deployment on it

- Describe your rollback story: when the verification step fails — or a bad version reaches production anyway — what happens, and how much of it is automated?

If running the pipeline end-to-end isn't feasible locally, it's fine for parts to be defined-but-demonstrated (e.g. run the CI job with `act`, or trigger the deploy script manually) — just make sure each stage is real, runnable, and documented.

Part 4 — Observability

Instrument the application and set up monitoring.

- Add application-level metrics to the code (think: request rate, error rate, latency, resource usage)
- Deploy a monitoring stack within your cluster — we suggest Prometheus + Grafana, but you're free to use any tooling you prefer
- Create at least one dashboard that answers: **"Is this service healthy right now?"**
- Define at least one meaningful alert rule — it doesn't need to actually send notifications, just be defined and ready to fire. Briefly justify each threshold, and note one thing you deliberately chose NOT to alert on and why

Part 5 — Automation: Zero to Running in One Command

We like solving things once. Package everything above so that a new engineer (or CI runner) can go from a clean machine to a fully running, monitored deployment with a single entry point.

- Provide one command (Makefile target, script, or small CLI — your choice) that brings up the cluster, deploys the full stack, and installs the monitoring
- It should be idempotent — safe to re-run — and fail loudly and clearly when a prerequisite is missing
- Include a teardown path

Bonus (optional, pick at most one)

Only if you have time left — we'd rather see the five parts done well.

- **Progressive delivery:** canary or blue/green rollout for PokeProxy (e.g. Argo Rollouts) with an automated analysis/abort step
- **Infrastructure as Code:** provision the local cluster and its addons via Terraform/OpenTofu instead of imperative scripts
- **AI tooling:** build a small agent-facing tool for this repo — e.g. a Claude Code skill, MCP server, or CLI that automates a repetitive task from this assignment (debugging a failed deploy, generating a rules.json, summarizing pipeline failures)

On Using AI

We encourage you to use AI tools (Claude, ChatGPT, Copilot, etc.) throughout this assignment. This reflects how we actually work — AI is part of the modern DevOps toolkit, and knowing how to leverage it effectively is a skill we value.

What matters to us is not whether you used AI, but whether you understand and can stand behind every decision in your submission. We will ask about your choices.

Deliverables

Please deliver:

1. **Fixed and hardened application code** — bugs fixed, structured logging added, observability and operability improvements applied
2. **Issue documentation** — for each bug/issue found, a write-up describing the problem, its production impact, and your solution
3. **Dockerfile(s)** for the application
4. **Kubernetes manifests** (or Helm charts) for the full deployment
5. **CI/CD pipeline** — pipeline definition(s), the post-deploy verification, and your rollback story
6. **Monitoring configuration** — metrics instrumentation, dashboard(s), alert rule definition(s) with threshold justifications
7. **One-command bootstrap** — the automation that stands everything up from scratch, plus teardown

8. **A README** explaining the layout of your submission and how the one-command bootstrap works
9. **Planning artifacts** — for each part, include your planning process: how you approached the problem, what options you considered, and why you made the choices you did. If you used AI tools, include the prompts or conversation flow that shaped your work. We care about your thinking as much as the output.

What Are We Looking For

We want to see how you think about shipping and operating production systems. Can you look at a service and identify what would break at 3 AM? Can you build the pipeline that gets a fix there in minutes, the automation that means nobody does it by hand twice, and the monitoring that tells you it worked? Do you make thoughtful choices, or just follow templates?

We don't expect perfection in every area. If you run out of time, document what you would do next and why. That tells us as much as the implementation itself.

Questions?

If anything is unclear, reach out to your hiring contact. We'd rather answer a question than have you guess at requirements.

The choice of environment and tools throughout the assignment is entirely up to you — local Kubernetes distribution (Minikube, kind, k3s, etc.), base images, Redis deployment strategy, networking approach, CI/CD tooling, monitoring stack, and anything else. Use what you're comfortable with and what you think makes sense. If you pick something, be ready to explain why.

[devops-home-assignment.zip](#)